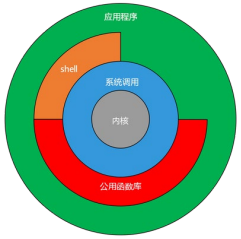
2024级计科操作系统
群号：1019499732

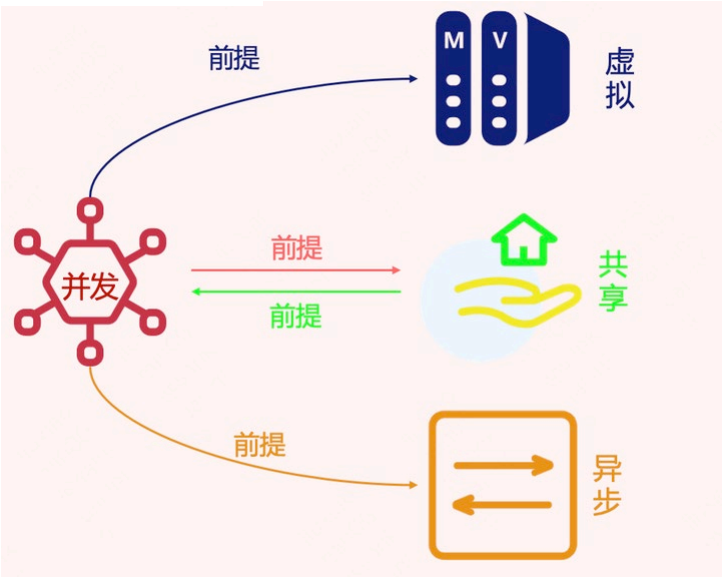
扫一扫二维码，加入群聊



武汉大学计算机学院

计算机科学与技术系

QQ



操作系统

主讲教师：蔡朝晖

武汉大学
Wuhan University

Linux

操作系统课程概述

- 操作系统课程内容
  - 操作系统的概念、演化历史和分类、操作系统的结构、内核功能及其实现原理、技术和方法。
- 课程考试方式及成绩计算
  - 闭卷考试
  - 平时成绩40%，其中：

    课后作业10%、实验20%、期中考试10%
  - 期末考试成绩60%

教材

- Abraham Silberschatz. Operating System Concepts (10th Edition), Wiley, 2018.
- 亚伯拉罕 希尔伯沙茨等著，郑扣根等译，操作系统概念（原书第10版），机械工业出版社，2023.9

OS参考书

- Operating Systems: Three Easy Pieces https://pages.cs.wisc.edu/~remzi/OSTEP/
- 操作系统导论（ Operating Systems: Three Easy Pieces ），王海鹏译，人民邮电出版社，2019.6
- 《现代操作系统：原理与实现》，陈海波等，机械工业出版社，2024.3
- 《深入理解计算机系统（原书第3版）》， Randal E. Bryant etc.著，龚奕利，贺莲译. 机械工业出版社，2021.
- 《计算机操作系统》，郑鹏等，武汉大学出版社，2022.3。第3版
- 《现代操作系统》英文影印版第4版，Andrew S. Tanenbaum著，机械工业出版社，2017.10
- 《操作系统：精髓与设计原理》英文版第9版，William Stallings，电子工业出版社，2020.6

课程内容

第1章 操作系统基础知识

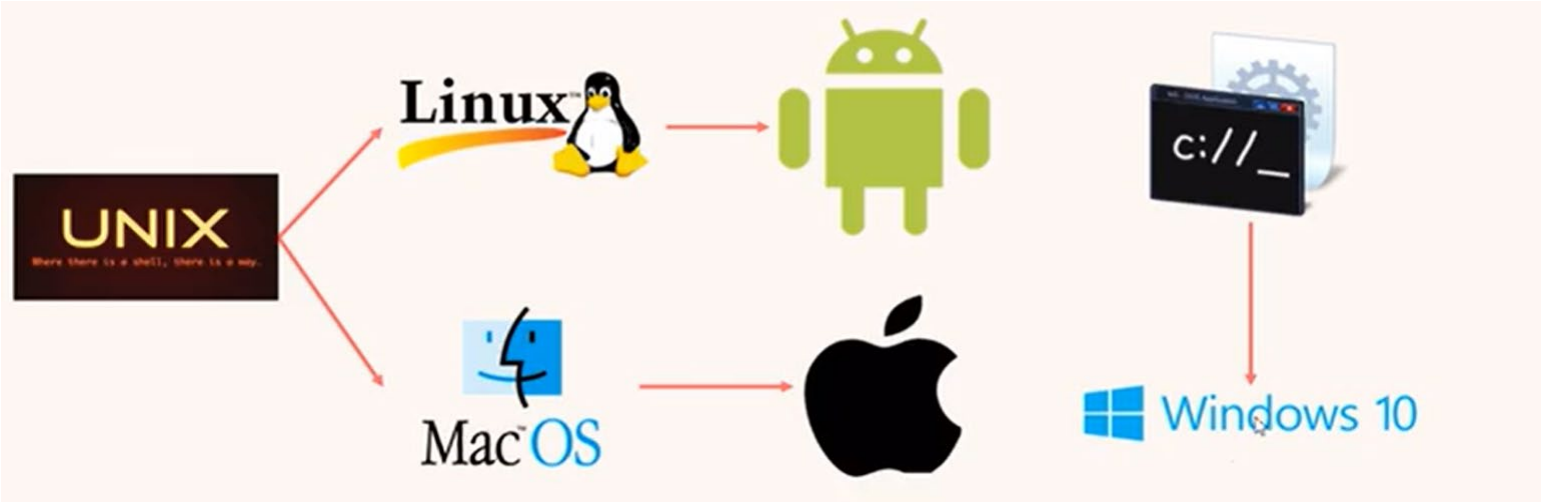
- 本章内容
  - 1.操作系统的定义、作用、主要特征和主要功能；
  - 2.操作系统的历史演化，典型的软硬件技术及典型操作系统；
  - 3.操作系统的分类；
  - 4.操作系统的结构及典型实例；
  - 5.操作系统的引导方式及引导过程，操作系统的启动机制和原理，程序执行的典型流程。

问题

- 什么是操作系统
- 操作系统的发展历史
- 操作系统面临的挑战
- 为什么要学习操作系统

什么是操作系统？

- 以下哪些是操作系统？
  - Windows 10所包含的所有软件
  - Linux内核以及所有设备的驱动
  - 在Macbook上下载安装的第三方NTFS文件系统
  - 华为Mate 30出厂时所有的软件
  - 大疆无人机出厂时所有的软件



操作系统与各组件的关系

- 用户
  - 人、机器、其它计算机
- 应用程序
  - 浏览器、办公软件、数据库系统、游戏
- **操作系统**
  - **控制和协调不同的用户和应用程序使用硬件**
- 硬件
  - CPU、内存、I/O设备

操作系统是管理硬件资源、控制程序运行、改善人机界面和为应用软件提供支持的一种系统软件。

从hello world说起

```c
#include <stdio.h>

int main(){
  printf('Hello World!');
  return 0;
}
```

```
#编译Hello World程序
$gcc hello.c -o hello

#运行一个Hello World程序
$ ./hello

#同时启动两个Hello World程序
$ ./hello & ./hello

Hello World!
Hello World!
```

hello运行时发生了什么？

操作系统有什么作用？

从操作系统的角度思考问题

- hello可执行文件如何存储在计算机中？
- 如何根据字符串hello找到存储在计算机中的文件？
- hello可执行文件如何加载到内存并在CPU上运行？
- hello执行文件如何将“hello world”显示在屏幕上？
- 两个hello程序是如何同时在一个CPU上运行的？
- 。。。

操作系统的作用

- 让系统易于使用，且正确高效地运行
  - 让程序能够与设备交互
  - 对资源调度进行共享调度
  - 允许运行多个程序，并解决引起的问题。
  - 外部接口多样化
- **研究操作系统的两个视角**
  - 从用户视角看：操作系统是用户与计算机硬件之间的接口。为用户使用计算机提供服务。
  - 从资源视角看：操作系统管理硬件资源且对硬件进行抽象。

操作系统的作用

- 管理与配置内存
- 决定系统资源供需的优先次序
- 控制输入设备与输出设备
- 操作网络与管理文件系统等基本事务
- 提供一个让用户与系统交互的操作界面

配置操作系统的主要目的

- 提供用户与计算机之间的接口，使计算机更易于使用；
- 有效地控制和管理计算机系统中的各种资源，使之得到更有效的利用；
- 合理地组织计算机系统的工作流程，以提高资源利用率并改善系统性能。
- **用户目标**：易用易学、可靠、安全、快速
- **系统目标**：易于设计、实现和维护，灵活、可靠、正确高效率

操作系统的设计目标

- 有效性
  - 提高系统资源利用率
  - 提高系统的吞吐量
- 方便性
- 可扩充性
- 开放性

- 管理系统资源
- 方便用户使用
- 作为扩充机器

操作系统的功能

- 管理计算机系统的软硬件资源
  - 处理机 —— 进程管理
  - 存储器 —— 内存管理
  - 设备 ——设备管理
  - 文件 —— 文件系统管理
- 用户与计算机“硬件资源”之间的接口
- 实现对计算机资源的抽象

进程管理

- 进程是正在**执行**的程序。
- 为了完成任务，进程需要一定的**资源**，包括CPU时间，存储器，文件，以及I/O设备
- 进程管理的主要任务是对CPU的使用实施有效的管理。
  - **进程控制**：负责进程的创建、撤消及状态转换
  - **进程同步**：对并发执行的进程进行协调。有同步与互斥
  - **进程通信**：负责完成进程间的信息交换
  - **调度**：决定哪个进程优先使用CPU

内存管理

- 内存可以看做是一个大数组，每个单元有自己的地址
- 内存可被CPU和I/O设备共享、可快速存取数据
- CPU执行的指令必须存放在内存中
- 内存管理功能
  - 按需**分配**和**收回**内存空间
  - **内存扩充**：逻辑上的扩充，虚拟内存
  - **内存保护**：彼此隔离

设备管理(I/O系统)

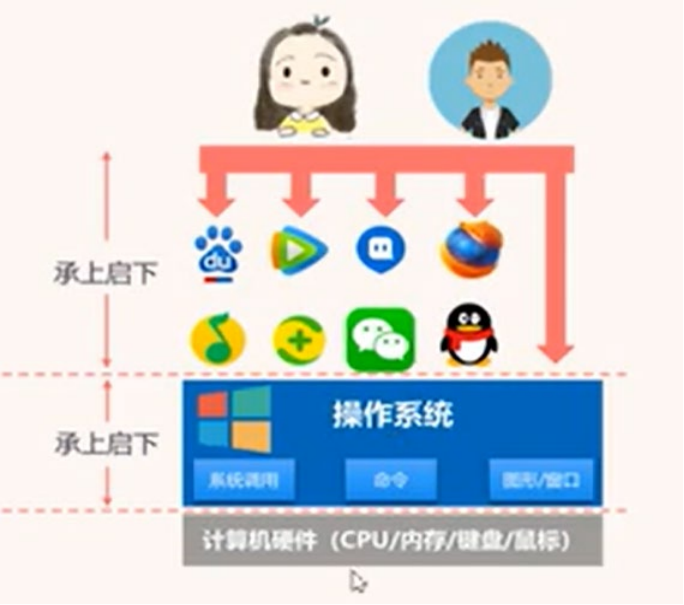
- 为用户隐藏具体硬件设备的特性
- 设备管理的功能
  - **设备分配**：根据用户的I/O请求，为之分配所需的设备，设备使用完成后还应回收
  - **缓冲管理**：对各类设备缓冲区进行有效管理
  - **设备驱动**：主要完成设备启动、I/O操作及中断处理
  - **设备独立性**：又称设备无关性，是指用户程序中的设备与实际使用的物理设备无关

文件管理

- 文件是由其创建者定义的一组相关信息的集合
- 操作系统文件管理的功能
  - **文件存储空间的管理**：对文件存储空间进行管理，包括存储空间的分配与回收等功能。
  - **目录管理**：管理文件的数据结构，提供按名存取的功能。
  - **文件操作管理**：从外存读入数据或将数据写入外存
  - **文件保护**：防止未授权用户存取文件；防止授权用户以不正确方式存取文件

操作系统的功能

- OS作为用户与计算机“硬件系统”接口
  - 程序接口
  - 命令接口
  - GUI(Graphical User Interface)，图形用户接口

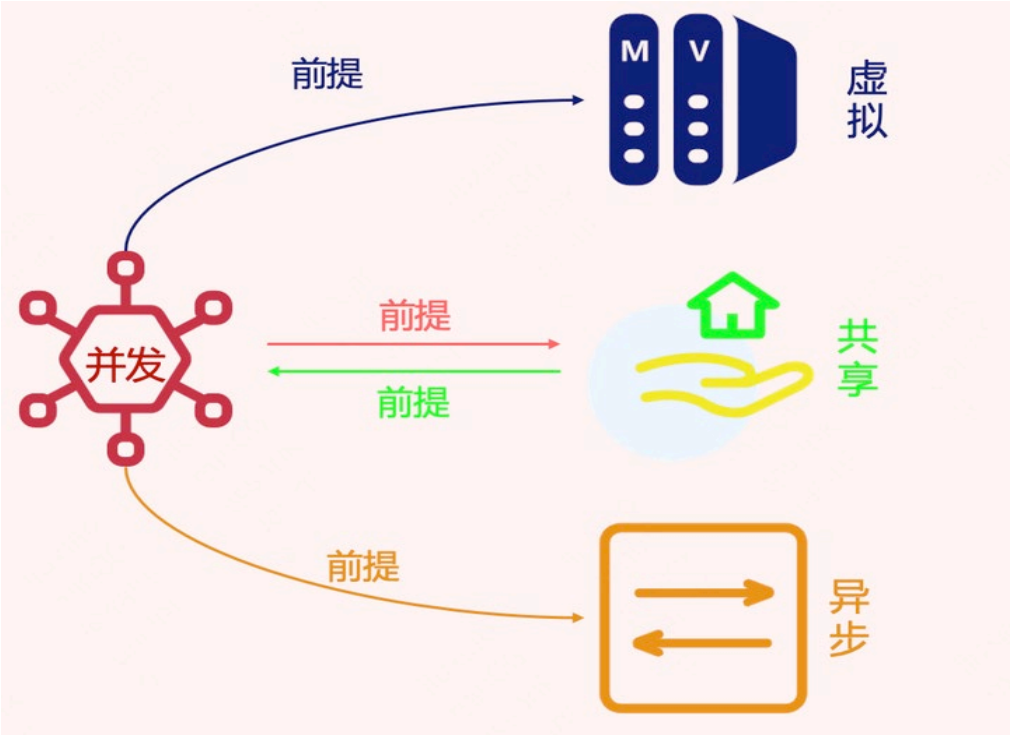


操作系统的功能

- OS实现了对计算机资源的抽象
  - 将具体的计算机硬件资源抽象成软件资源方便用户使用
  - 开放了简单的访问方式，隐藏了实现细节
  - 举例:使用浏览器进行搜索

操作系统的主要特征

- OS的四个基本特征
  - 并发(Concurrency)
  - 共享
  - 虚拟
  - 异步



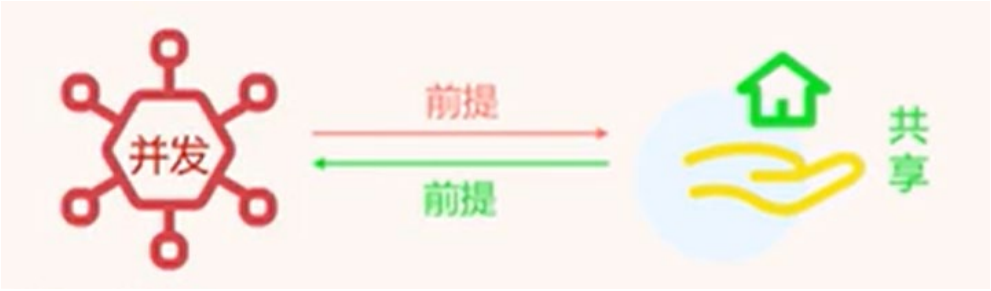
操作系统的主要特征

- **1.并发：** 在同一时间间隔内发生两个或多个事件
  - 相似概念：并行：同一时刻发生多个事件

- **2.共享：** 系统中的资源可供多个并发执行的进程共同使用
  - 互斥共享：一段时间只允许一个进程访问
  - 同时访问：一段时间允许多个进程访问
  - 时分复用、空分复用

操作系统的主要特征

- 并发与共享互为存在条件
  - 程序的并发执行带来资源共享问题；
  - 系统对资源共享的管理影响到程序并发执行的效率
  - 如果多道程序可以共享部分资源（比如磁盘），才能并发。



操作系统的主要特征

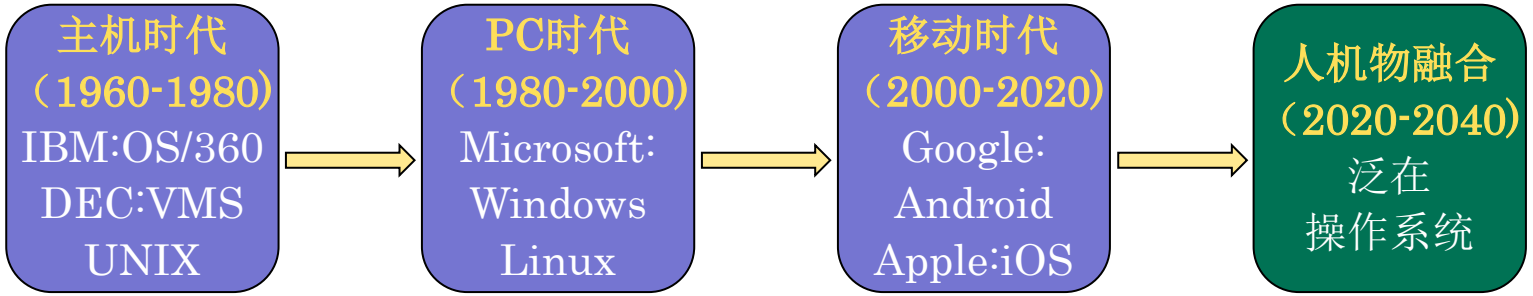
- **3.虚拟**
  - 把一个物理上的实体变为若干个逻辑上的对应物
  - 分时技术、虚拟内存、虚拟设备
- **4.异步（不确定）**
  - 任务的执行顺序和每个任务的执行时间是不确定的
    - 获得运行的时机
    - 因何暂停
    - 每道程序需要多少时间
    - 不同程序的性能，比如计算多少，I/O多少

虚拟化

- 虚拟化（virtualization）技术
  - 将物理资源转换为更通用、更强大且易于使用的虚拟形式
  - 虚拟化CPU
    - 进程/线程，数量不受物理CPU的限制
  - 虚拟化内存
    - 虚拟内存，大小不受物理内存的限制
  - 虚拟化I/O
    - 将各种设备统一抽象为文件，提供统一接口

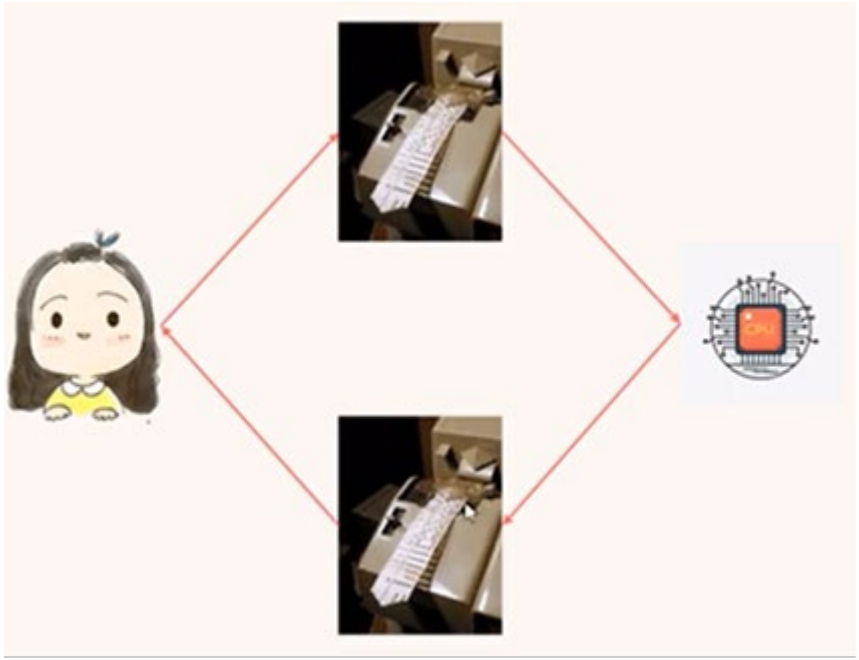
操作系统的历史演化

- 操作系统的发展过程是一个从无到有，从简单到复杂的过程。
- 最初计算机上无操作系统，20世纪50年代出现了简单批处理系统，60年代出现了多道批处理系统，不久又出现了分时系统及实时系统...

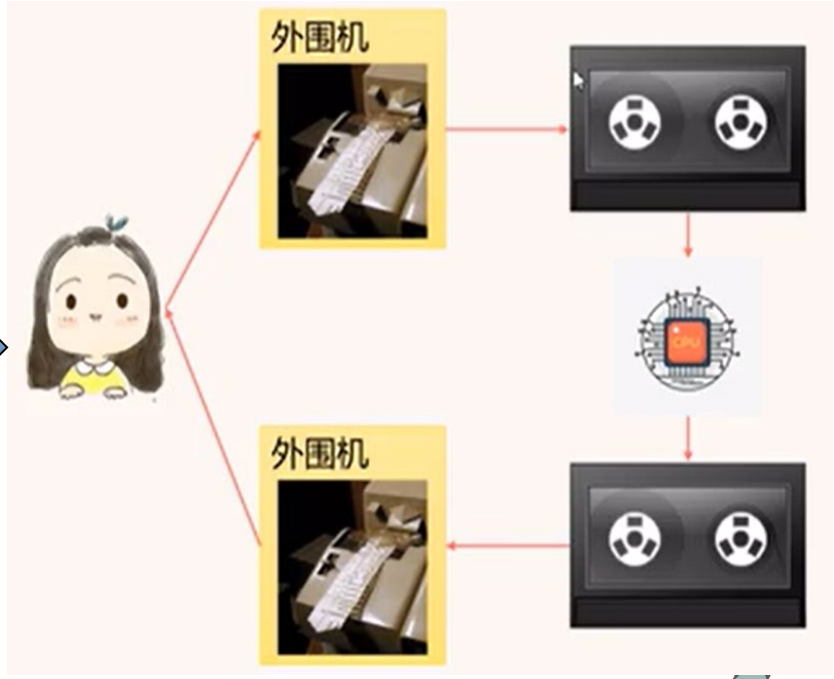


手工操作阶段

- 人工操作方式：46~50年代中后期，计算机系统上没有配置操作系统，人们使用计算机采用手工操作方式。



脱机

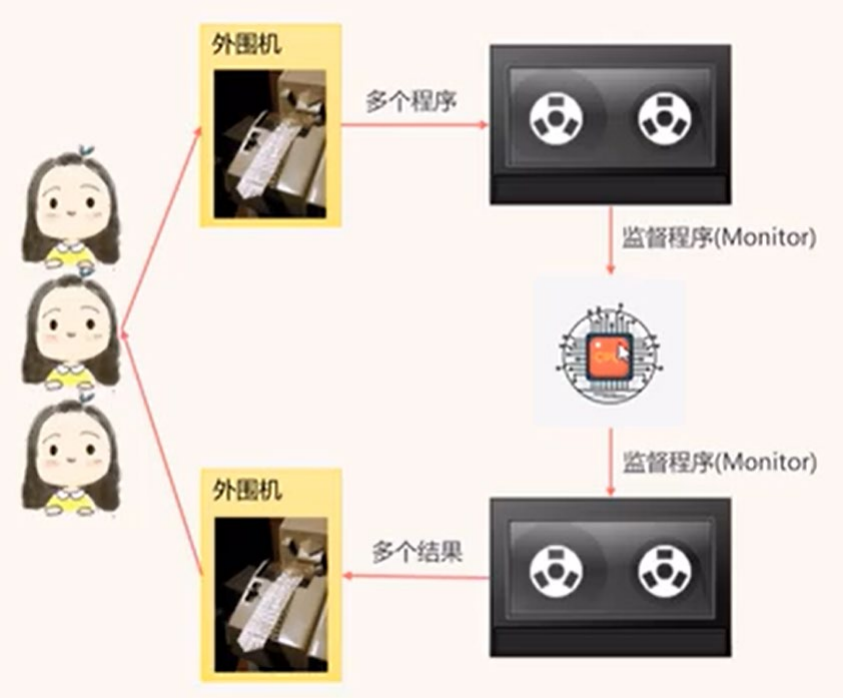


手工操作方式的特点

- 手工操作方式的特点：
  - 用户独占计算机资源，资源利用率低
  - CPU等待人工操作
- 手工操作方式的不足：
  - 手工操作的慢速与CPU运算的高速之间的矛盾，此即人机矛盾。
  - CPU的快速与I/O设备慢速的矛盾。

早期批处理

- 为解决人机矛盾，人们提出了从一个作业到下一个作业的自动过渡方式，从而出现了批处理技术。
- **批处理技术**是指计算机系统对一批作业自动进行处理的一种技术。

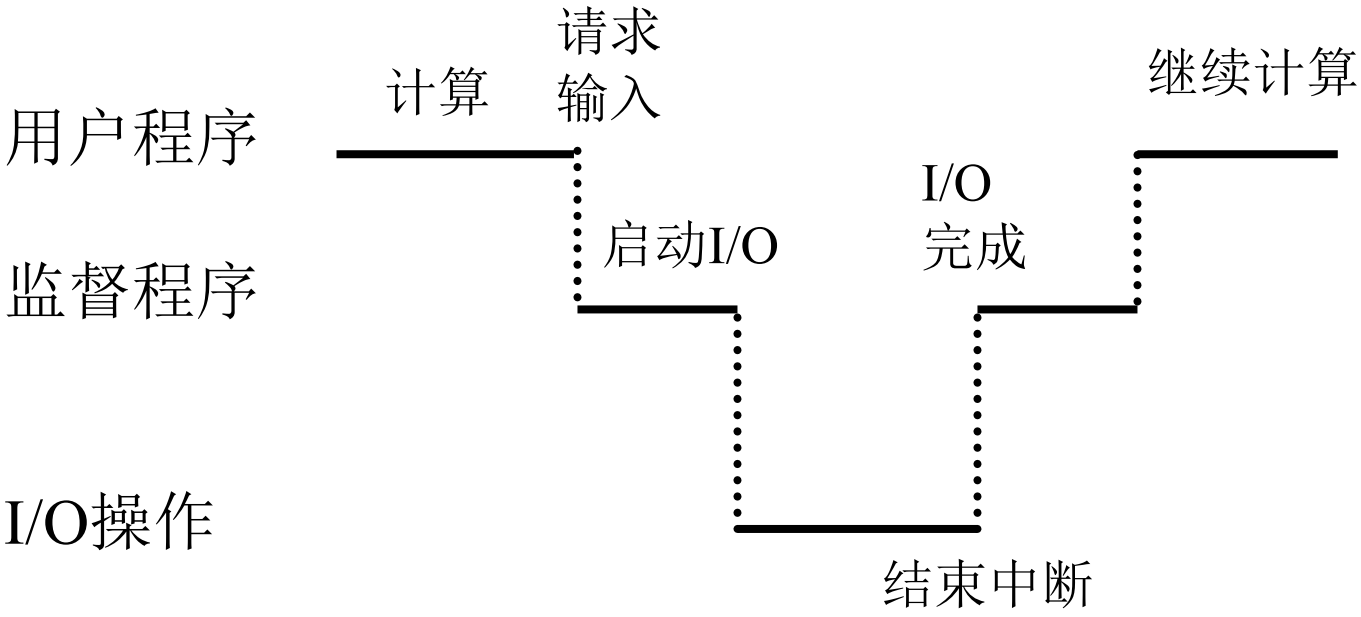


批处理技术

- **监督程序**是一个常驻内存的程序，它管理作业的运行，负责装入和运行各种系统程序来完成作业的自动过渡。
- 监督程序是最早的操作系统雏形。
- 批处理分为：
  - 单道批处理系统：自动性、顺序性、单道性
  - 多道批处理系统：

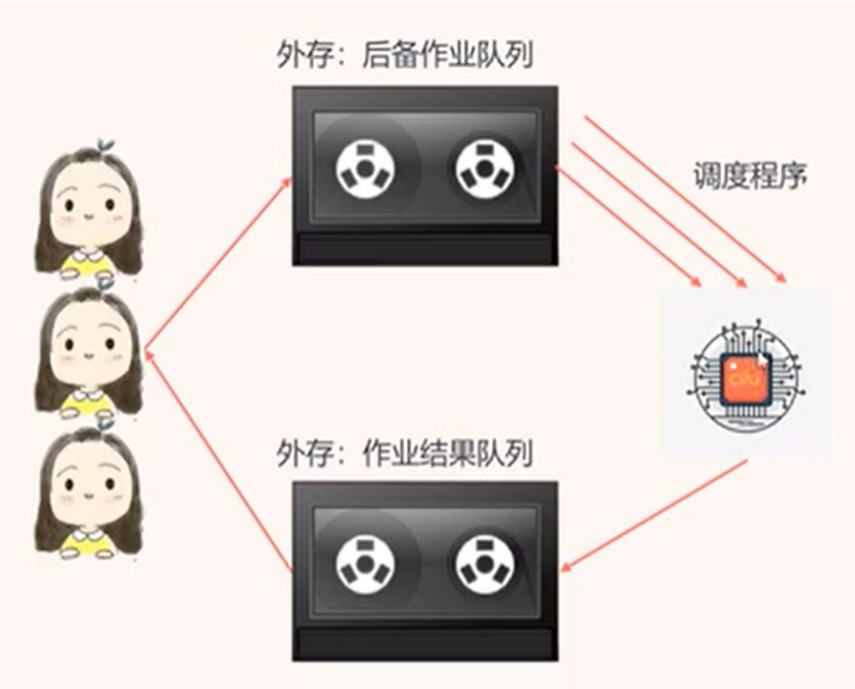
单道批处理系统

- 单道批处理系统中内存仅一道程序，系统资源无法得到充分利用。下图是单道程序运行实例

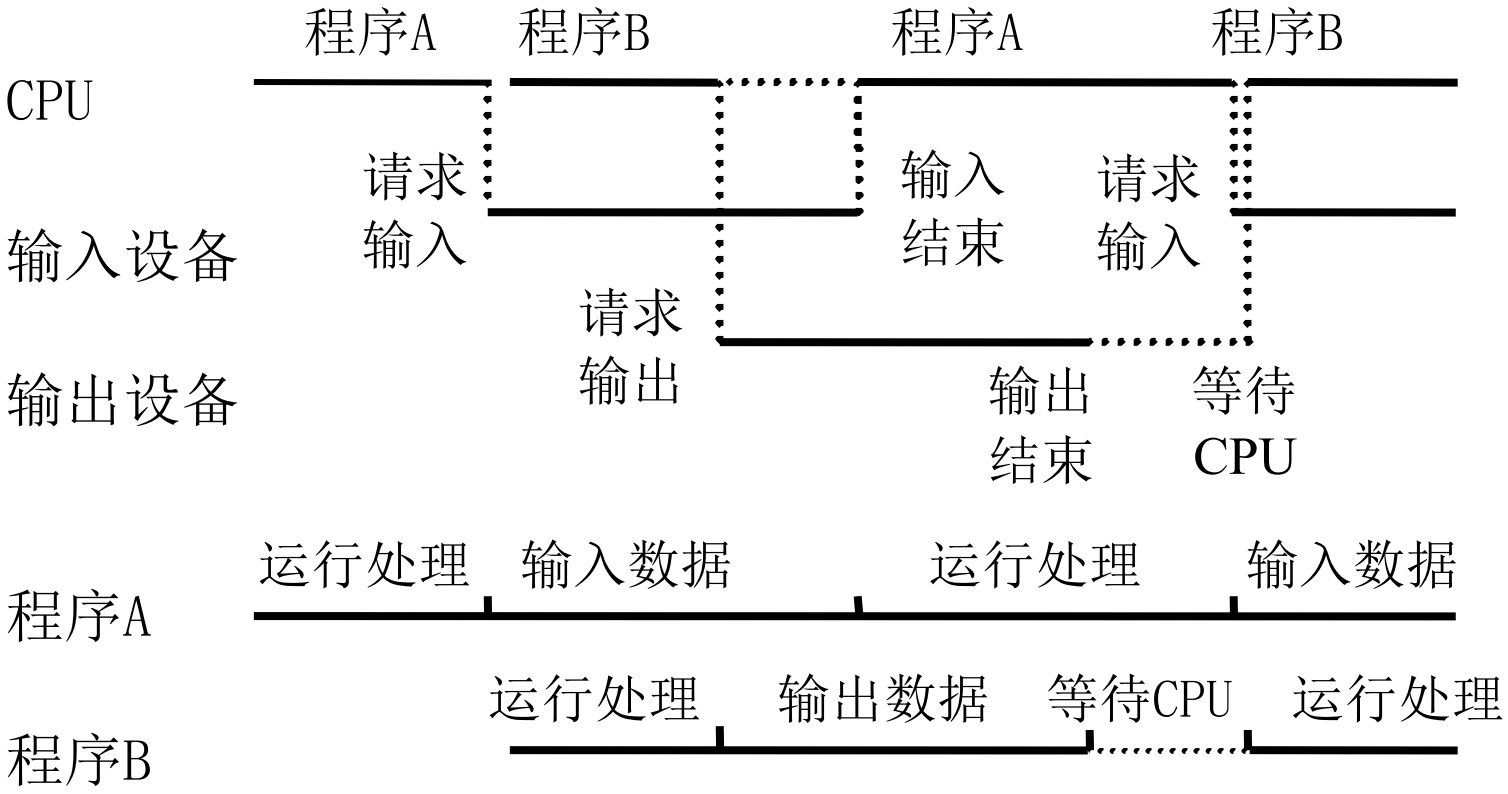


多道批处理系统

- 多道程序设计通过在内存中同时保存多道程序，让CPU在一个程序等待I/O时可以继续执行其他程序，从而显著提高了资源利用率
- 多个存储在主存中的程序，在管理程序的控制下交替运行，共享CPU和系统中的其他资源。



多道程序运行实例

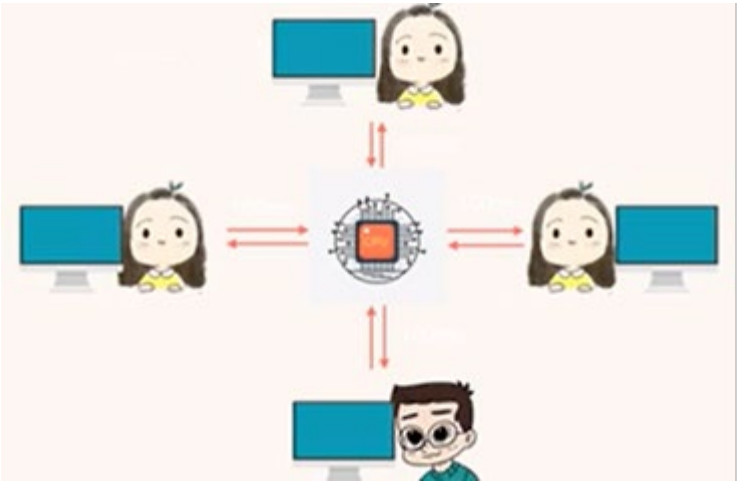

程序A
程序B
程序A
程序B
CPU
请求
输入
输入
结束
请求
输入
输入设备
请求
输出
输出设备
输出
结束
等待
CPU
运行处理
输入数据
运行处理
输入数据
程序A
运行处理
输出数据
等待CPU
运行处理
程序B

多道程序设计的特点

- 核心目标是提高CPU利用率，增加系统吞吐量。
- 多个程序并发执行
- 允许资源共享
- 有内存管理能力防止程序间互相影响
- 高效的任务调度与上下文切换机制
- 平均周转时间长
- 无人机交互

分时系统

- 它允许多个用户通过终端同时与计算机系统进行交互，每个用户都能以相对独立的方式使用系统资源。操作系统通过将CPU的时间分成细小的片段（称为时间片），并快速地在多个用户任务之间切换，给每个用户一种“独占”计算机资源的感觉，从而实现多用户的共享计算机资源。



分时系统的特点

- 多用户通过终端与主机互联
- 具有及时性和交互型
- 通过时间片轮转的方式使用CPU，短时间内可以获得响应
- 提供资源的共享与保护
- 缺点：用户优先级相同，不能优先处理紧急任务。

分时技术示意图

- 假设系统中有4个联机用户，按1、2、3、4的顺序排列：

时间片	运行	就绪队列
0		用户1 → 用户2 → 用户3 → 用户4
1	用户1	用户2 → 用户3 → 用户4
2	用户2	用户3 → 用户4 → 用户1
3	用户3	用户4 → 用户1 → 用户2
4	用户4	用户1 → 用户2 → 用户3
5	用户1	用户2 → 用户3 → 用户4

实时系统

- **实时系统**（Real-Time System）是一种计算机系统，它在**特定的时间约束下**执行任务，对外部事件作出**快速响应**。实时系统的核心特征在于其时间敏感性，即系统必须在规定的时间内完成某项任务或对某个事件作出响应，否则将导致系统功能失效或严重后果。实时系统通常用于那些对时间要求严格的应用场景，如工业控制、航空航天、军事系统、医疗设备和通信系统等。

实时系统的特点

- 时间约束（Time Constraints）
  - 硬实时系统（Hard Real-Time System）
  - 软实时系统（Soft Real-Time System）
- 确定性（Determinism）
  - 系统在相同条件下应始终给出相同的输出和行为
- 高可靠性和稳定性：
  - 系统必须设计得非常健壮，能够承受各种故障并具有快速恢复能力。
- 任务优先级（Priority Scheduling）
  - 确保关键任务能够及时完成
- 快速上下文切换（Fast Context Switching）
  - 快速地进行上下文切换，以保证高优先级任务能够尽快执行

操作系统的分类

- 按用户数量
- 按任务管理
- 按实时要求
- 按计算机规模和应用领域
  - 大型机OS、桌面OS、服务器OS、嵌入式OS
- 按操作模式
  - 批处理OS、分时OS、网络OS、分布式OS
- 按内核结构
- 按开放性和专有性
- 按市场定位和应用环境
  - 消费级OS、企业级OS

操作系统面临的挑战

- 安全性和隐私保护
- 多核和异构计算支持
- 虚拟化与云计算优化
- 物联网和边缘计算支持
- 能效与电源管理
- 快速适应硬件与软件变化
- 用户体验和界面的改进
- 集成人工智能与机器学习技术
- 可靠性与可用性保障
- 数据与存储管理的优化

操作系统的结构

- OS是一个复杂的大型软件
- 如何控制OS设计的复杂度？
  - 策略和机制相分离，
  - 策略：做什么？
  - 机制：怎么做？

	策略	机制
登录	什么用户，以什么权限登录等	输入处理、策略文件管理、桌面启动加载等
调度	先到先服务、时间片轮转等	调度队列的设计、调度实体的表示、调度的中断处理等

操作系统复杂度管理方法

- 管理复杂系统的重要方法：M.A.L.H
  - **模块化**：将复杂系统分解为一系列模块，通过明确定义的接口进行交互，高内聚、低耦合
  - **抽象**：接口与内部实现分离，模块通过抽象的接口进行交互，而无需关心模块的内部实现
  - **分层**：将模块按一定的原则进行层次划分，约束模块只能同层或与相邻层交互
  - **层级**：功能相近的模块组成具有清晰接口的自包含子系统，子系统再递归地组成有清晰接口的更大子系统

操作系统内核结构

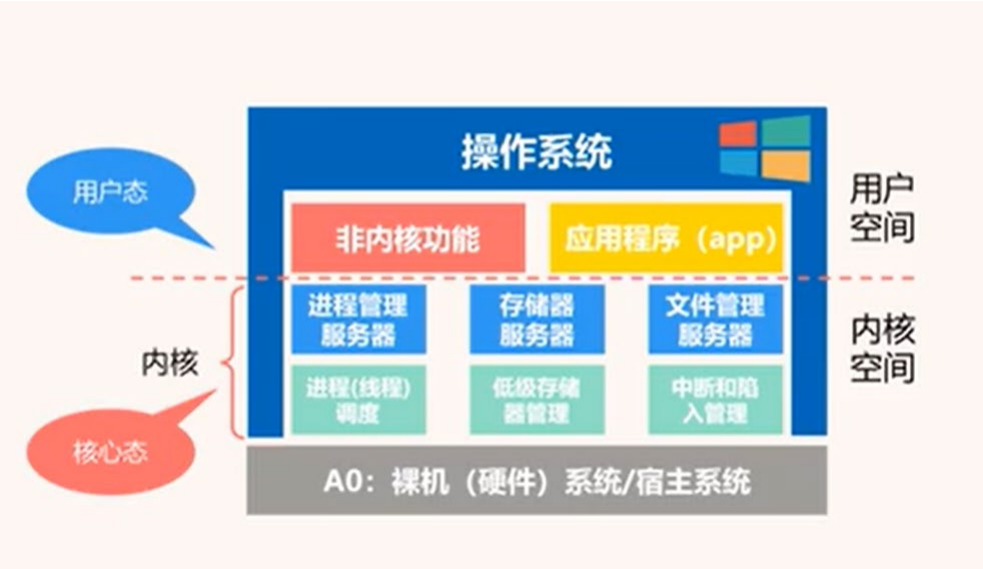
- 简单结构 —— MS-DOS
- 宏内核（单内核）—— UNIX/Linux
- 模块化结构
- 微内核 —— Mach

模块化结构

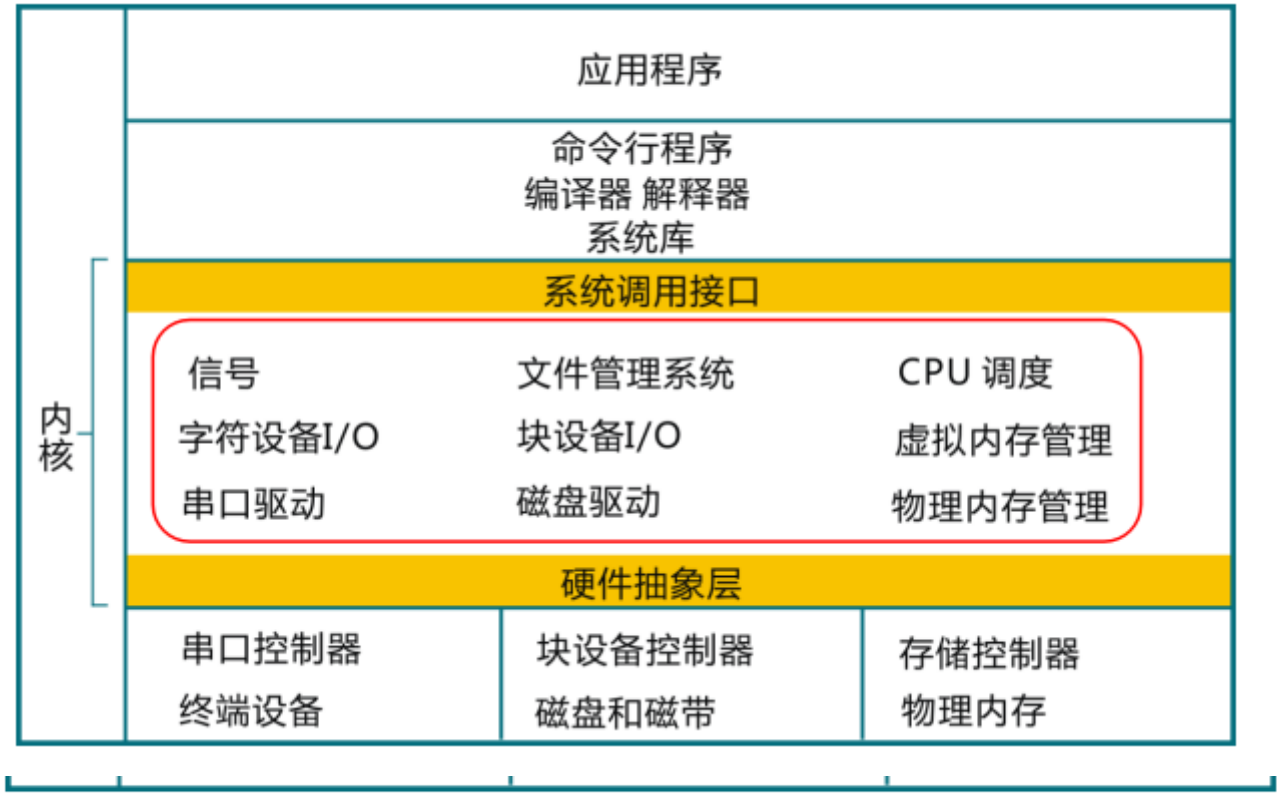
- 模块功能独立，且被封装
- 具有良好定义的接口
- 现代OS(如Unix、Linux、Windows等)均采用模块化的策略组织各功能
- 没有商业OS是纯粹采用模块化结构设计
- 优点：易于理解，效率高
- 缺点：全局函数使用多造成访问制困难，结构不清晰会导致可理性、可维护性及可移植性差，存潜在的性能退化

宏内核结构

- 有序分层法，自顶向下依次依赖
- 设计时，自底向上:每一步建立在可靠的基础上
- 优缺点:容易保证系统正确性容易扩充和维护自上而下的层次通信，导致系统效率降低

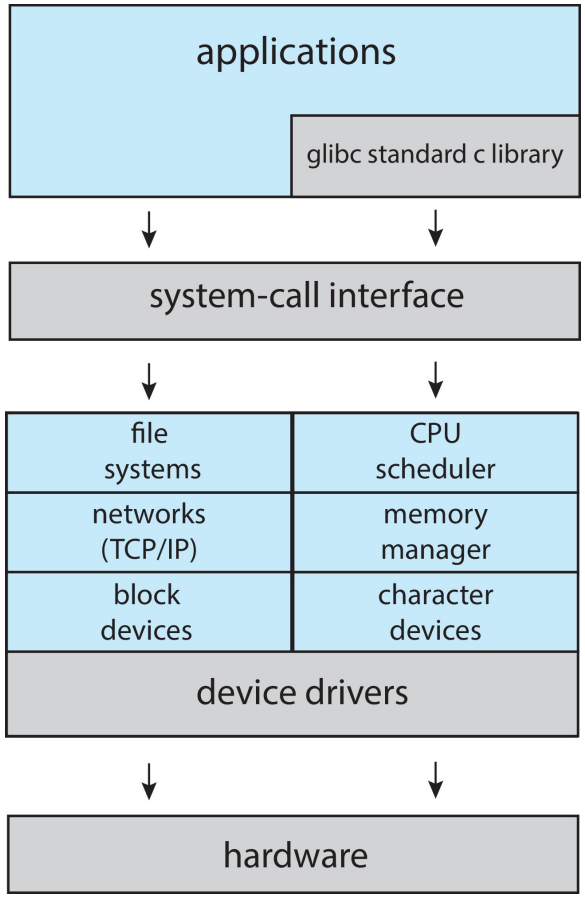


典型的UNIX操作系统



Linux系统结构

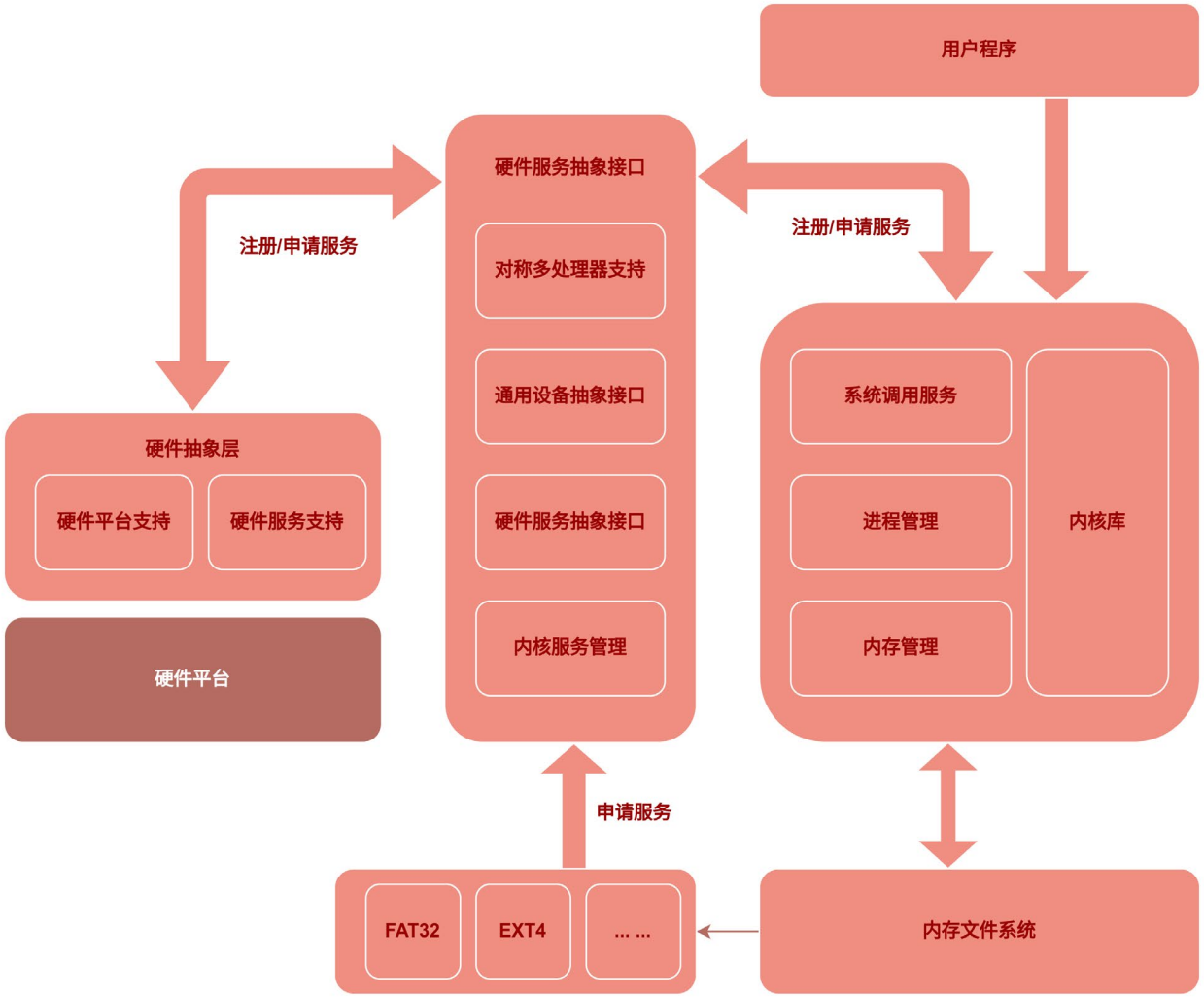
- 宏内核+模块化设计



宏内核的特点

- 优点：运行效率高
- 缺点：结构难以理解、难以维护，随着复杂性的增加，可靠性和安全性难以保障

WHUcore


用户程序
硬件服务抽象接口
对称多处理器支持
通用设备抽象接口
硬件服务抽象接口
内核服务管理
注册/申请服务
注册/申请服务
硬件抽象层
硬件平台支持
硬件服务支持
硬件平台
系统调用服务
进程管理
内存管理
内核库
申请服务
FAT32
EXT4
... ...
内存文件系统

微内核结构

- 微内核结构基于客户/服务器模型，由微内核和核外用户空间的服务器进程构成
- 微内核保留进程管理、内存管理和中断管理，
- 通过消息传递使客户程序与服务程序进行通信
- 优点
  - 易于扩展
  - 易于从一个硬件架构移植到另一个
  - 更可靠（内核代码更少）
  - 更安全
- 缺点
  - 用户空间到内核空间的通信增加了系统开销，早期性能不及宏内核结构

微内核结构

操作系统
非内核功能
应用程序（app）
进程管理服务器
存储器服务器
文件管理服务器
进程(线程)调度
低级存储器管理
中断和陷入管理
A0：裸机（硬件）系统/宿主系统
用户态
大内核
微内核
核心态

混合结构

- 现代OS往往采用的不是单一模型
  - 混合模型结合多种方法来满足性能、安全性和可用性需求
  - Linux和Solaris是单内核结构，但它们也是模块化的，可以动态向内核添加新功能
  - Windows是单内核结构，但也保留了微内核的模式来支持子系统，也提供可加载的内核模块
  - MacOS是混合、分层的系统，采用Mach微内核与部分BSD Unix构成内核，且有可动态加载的模块

操作系统的构建和启动

- OS可运行于各种不同的系统上
- 从源码开始构建OS
  - 1. 获取OS源码
  - 2. 为某系统即将构建的OS配置参数
  - 3. 编译OS
  - 4. 安装OS
  - 5. 启动计算机并运行OS
- 预编译的OS可能过于通用而无法支持特定的硬件

构建Linux操作系统

1. 从**http://www.kernel.org**下载Linux源代码。
2. 使用“**make menuconfig**”命令配置内核。这一步生成.config配置文件。
3. 使用“**make**”命令编译主内核。make命令根据.config文件中的配置参数编译内核，生成内核镜像文件vmlinuz。
4. 使用“**make modules**”命令编译内核模块。与编译内核类似，模块的编译也依赖于.config文件中指定的配置参数。
5. 使用“**make modules install**”命令将内核模块安装到vmlinuz中。
6. 使用“**make install**”命令将新内核安装到系统中。

系统启动

- 打开电源并加载内核的过程称为**系统启动**
- OS必须对硬件可用，这样硬件才能启动它
  - 存储在ROM或EPROM中的一段代码，即引导加载程序BIOS，将其载入内存并启动执行
  - 有时分两步执行，ROM中的代码BIOS加载固定磁盘位置上的引导块(boot block)，再由引导块从OS所在分区的特定位置读取引导加载器(bootstrap loader)，由它将OS内核从磁盘加载到内存
  - 现代OS使用UEFI来代替BIOS

OS的引导过程（滚雪球方式）

- 初始引导
  - 系统加电
  - 执行初始引导程序，对系统硬件和配置进行自检，保证系统没有硬件错误
  - 从硬盘中读入操作系统引导程序，并将控制权交给该程序模块
- 引导程序执行
  - 引导程序执行，将操作系统核心文件读入内存，并将控制交给核心的初始化程序。

OS的引导过程（滚雪球方式）

- 内核初始化，初始化系统数据结构及参数
  - 系统加电建立进程有关的数据结构；
  - 获得自由存储空间的容量，建立存储管理的数据结构；
  - 建立系统设备和文件系统的数据结构；
  - 初始化时钟。
- 系统初始化
  - 完善OS的操作环境，装载命令处理程序 (或图形用户界面)，并初始化；
  - 在多用户系统中，为每个终端建立命令解释进程，使系统处于命令接收状态。

OS的启动流程

1. 引导加载程序加载OS内核到内存中。
2. 内核初始化：OS内核进行一系列的初始化操作，如硬件设备检查，加载驱动，内存管理等。
3. 内核启动init进程：内核初始化完成后，会启动init进程（在Linux中）或systemd进程，该进程是所有其他用户进程的父进程。
4. 用户空间程序启动：init进程根据系统的配置，启动一系列的用户空间程序，如Shell，图形界面等。

XV6启动代码（risc_v）

1. 当操作系统启动时，CPU将会重置到一个预定义的物理地址（即0x80000000），从而开始运行entry.s。这个文件确定栈指针地址，对每一个CPU核分配对应的栈空间，之后跳转到start.c进行接下来的操作。
2. 在系统跳转到start.c后，start.c对于存储器以及中断进行管理。例如禁止分页等。在此之后，start.c将mepc设为main函数的地址，并使用mret功能跳转到main函数，进行下一步的操作。
3. CPU跳转到main函数中。对于多核CPU，其0号核负责所有模块的初始化操作。之后，系统根据当前核的不同输出不同的提示信息。最后所有核所运行的线程均进入死循环。

XV6启动代码

- entry.s

```
.section .text
.global _entry
_entry:
        # set up a stack for C.
        # stack0 is declared in start.c, # with a
        4096-byte stack per CPU. # sp = stack0 +
        (hartid * 4096)
        la sp, stack0
        li a0, 1024*4
        csrr a1, mhartid
        addi a1, a1, 1
        mul a0, a0, a1
        add sp, sp, a0
        # jump to start() in start.c
        call start
spin:
        j spin
```

Linux的启动

- Linux采用滚雪球的方式启动
  - 加电或复位 → BIOS的启动 → Boot Loader → OS初始化

①系统加电或复位

- 对主存中所有的数据清零，对内存进行校验，若无错，

  CS：IP → BIOS入口。

Linux的启动

② BIOS启动

- 在ROM中的引导程序放在固定位置：FFFF：0000　CPU从这里开始执行。
  - 上电自检；
  - 对硬件设备进行检测和连接，并将测得的数据送入BIOS数据区；
  - 从盘中读入Boot Loader (引导程序)。
- 从硬盘启动时，读入零柱面零磁道1扇区MBR (Master Boot Record)，将控制权交Boot Loader。

Linux的启动

③ Loader(引导程序)

- 功能：将OS读入内存，并将控制权交给OS的初始化程序。

④ 系统核心初始化(Setup.s)

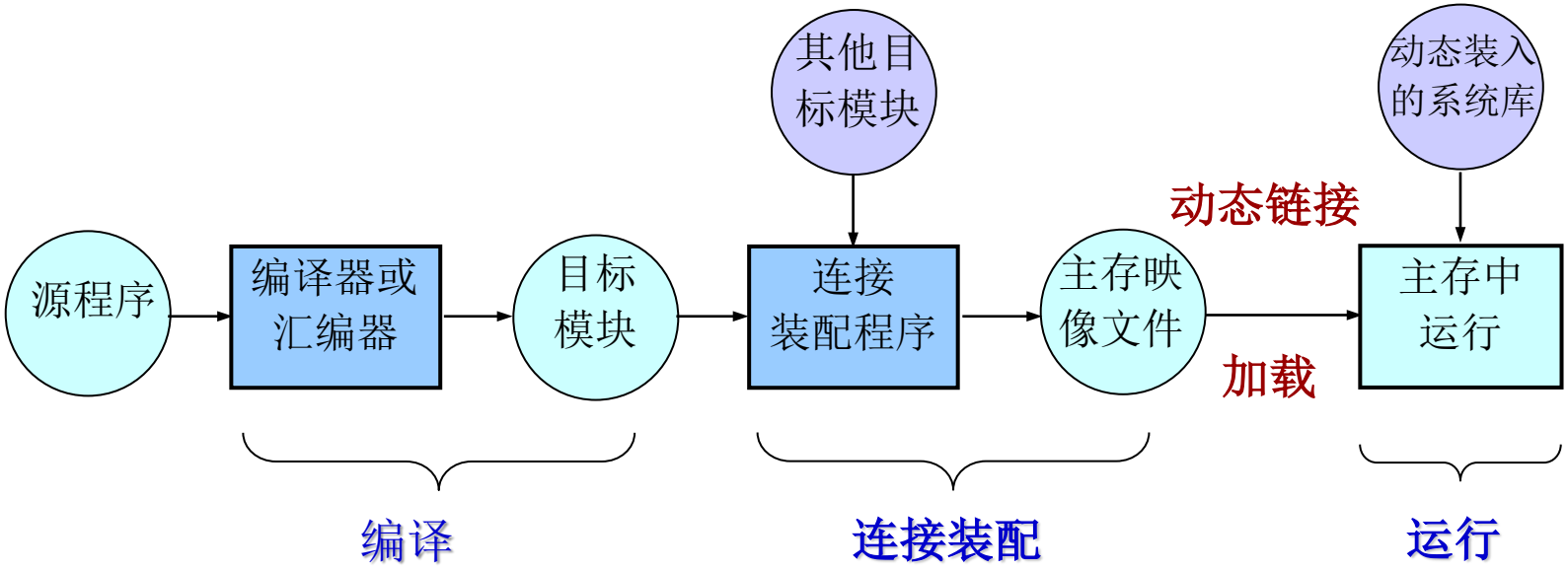
- Setup的工作
  - 检查调入内存中的代码；
  - 获取内存容量信息，设置设备模式；
  - 屏蔽中断，准备进入保护模式；
  - 设置中断描述符表 (idt)，全局描述符表 (gdt)；控制权交给 Heads。
- Heads的工作
  - 对中断向量表作准备工作；
  - 检查CPU类型；
  - 调用Setup_paging进行页面初始化；
  - 调用main.c中的Start_kernel()。

Linux的启动

- Start_kernel()的工作
  - 对与CPU、内存等最基本硬件相关部分进行初始化；
  - 对中断向量表进行初始化；
  - 为进程调度程序作准备；
  - 设置基准时钟；
  - 内核的主存分配；
  - 对文件系统进行初始化；
  - 建立init进程。

init进程对每一个联机终端建立“getty”进程，getty在端上显示“login”，等待用户登录。

应用程序的处理步骤



应用程序处理步骤示意图

应用程序的处理步骤

- 编辑
  - 建立一个新文件，或对已有的文件中的错误进行修改。
- 编译
  - 将源程序翻译成浮动的目标代码。
- 连接
  - 主程序和其他所需要的子程序和例行程序连接装配在一起，使之成为一个可执行的、完整的主存映像文件。
- 运行
  - 将主存映像文件调入主存，启动运行，得出计算结果。

一个程序的典型执行流程

1. **加载：** 当启动一个程序时，由OS的加载器将该程序的可执行文件加载到内存中。
2. **运行：** OS创建一个新的进程，并将CPU的控制权交给该进程。该新进程开始执行加载到内存中的程序代码。
3. **系统调用：** 当程序需要进行一些特权操作（如读写文件，发送网络数据等）时，它将发起系统调用。
4. **中断和信号：** 程序的执行可能会被中断和信号打断。中断通常是由硬件事件触发的，如I/O操作的完成，定时器的超时等。信号则是一种软件中断，可以由其他进程或者内核发送。
5. **退出：** 当程序执行完成或者由于某种原因需要停止时，它将执行退出操作，包括释放资源，关闭打开的文件，通知父进程等。

操作系统的运行(Operation)

- 引导程序→加载内核→启动系统守护进程
- 内核是中断驱动的（硬件和软件）
  - 由任一设备触发硬件中断
  - 软件中断（异常或陷阱）
    - 软件错误（例如，除数为零）
    - 请求操作系统服务 —— 系统调用
    - 其他进程问题包括无限循环，进程相互修改或修改操作系统

OS对程序执行的支持

- OS如何让程序能在计算机上顺利执行？
  - 装载并运行程序
  - 管理内存空间：为程序提供虚拟地址空间，实现逻辑地址到物理地址的转换
  - 支持输入/输出
  - 提供错误处理和保护

为什么要学习操作系统？

- 理解计算机的基本工作原理
- 提高编程和软件开发能力
- 适应现代科技和发展趋势
- 增强系统安全意识和技能
- 适应不同的职业需求
- 培养计算机科学思维和能力
- 适应科技发展的快速变化
- 。。。

练习题

- 1.2 We have stressed the need for an operating system to make efficient use of the computing hardware. When is it appropriate for the operating system to forsake this principle and to “waste” resources? Why is such a system not really wasteful?
- 1.4 Keeping in mind the various definitions of operating system, consider whether the operating system should include applications such as web browsers and mail programs. Argue both that it should and that it should not, and support your answers.

练习题

- 1.5 How does the distinction between kernel mode and user mode function as a rudimentary form of protection (security)?

- 1.6 Which of the following instructions should be privileged?
  - a. Set value of timer. b. Read the clock.
  - c. Clear memory. d. Issue a trap instruction.
  - e. Turn off interrupts. f. Modify entries in device-status table.
  - g. Switch from user to kernel mode.
  - h. Access I/O device.

练习题

- 请简述计算机从开机到操作系统完全启动的基本过程？
- 什么是引导扇区（Boot Sector）？它在操作系统的启动过程中起什么作用？
- 请描述一个程序从点击图标到完全运行的典型执行流程？

选择题1

- 在脱机批处理方式中，有一台负责与外部设备交换信息的计算机，一般称之为_____。

  A. 终端处理机　　B. 外围处理机

  C. 客户机　　D. 服务处理机

- 在计算机系统中，操作系统是_____。

  A. 一般应用软件　　B. 核心系统软件

  C. 用户应用软件　　D. 硬件

选择题2

- 实时操作系统必须在_____内处理来自外部的事件。

  A.时间片　　B.周转时间

  C.被控制对象规定时间　　D.一个机器周期

- 在设计实时操作系统时，不重点考虑的是_____。

  A．及时响应，快速处理　　B．有高安全性

  C．提高系统资源的利用率　　D．有高可靠性

选择题3

- 下述关于并发性的叙述中正确的是_____。

  A.并发性是指若干事件在不同时间间隔内发生

  B.并发性是指若干事件在同一时间间隔内发生

  C.并发性是指若干事件在不同时刻发生

  D.并发性是指若干事件在同一时刻发生

- 分时系统追求的目标是_____。

  A.快速响应用户　B.充分利用I/O设备

  C.充分利用内存　D.提供系统吞吐率

选择题4

- 一个多道批处理系统，提高了计算机系统的资源利用率，同时_____。

  A. 减少各个作业的执行时间

  B. 增加了单位时间内作业的吞吐量

  C. 减少了部分作业的执行时间

  D. 减少单位时间内作业的吞吐量

- 批处理系统的主要缺点是_____。

  A. 无交互能力　　B. 系统吞吐量小

  C. 资源利用率低　　D. CPU利用率不高

选择题5

- 从用户的观点看，操作系统是_____。

  A.合理地组织计算机工作流程的软件

  B. 由若干层次的程序按一定的结构组成的有机体

  C. 控制和管理计算机资源的软件

  D.用户与计算机之间的接口

- 所谓_____是指将一个以上的作业放入内存，并且同时处于运行状态，这些作业共享处理机的时间和外围设备等资源。

  A.多道程序设计　　B.多重处理

  C.共行执行　　D.实时处理

选择题6

- 操作系统中最基本的两个特征是_____。

A. 虚拟和不确定　　B.共享和虚拟

C. 并发和共享　　D.并发和不确定